

Otkrivači grešaka u distribuiranim sustavima

Bartol Gracin Vid Joha Matija Njegač

Distribuirani procesi

Pregled

1. Motivacija: asinkroni sustavi i nemogućnost konsenzusa
2. Otkrivači grešaka: potpunost i točnost
3. Osam klasa i reducibilnost
4. Konsenzus pomoću otkrivača ($\mathcal{S}$ i $\Diamond\mathcal{S}$)
5. Implementacije:
 - Algoritam 15.3 — konsenzus uz $\mathcal{S}$
 - Algoritam 15.4 — konsenzus uz $\Diamond\mathcal{S}$
 - Algoritam 15.7 — otkrivač $\Diamond\mathcal{P}$

Motivacija

- **Asinkroni model:** nema gornje granice na trajanje koraka ni na kašnjenje poruke.
- Proces može zakazati *zaustavljanjem* (crash) — trajno stane.
- Problem: ne razlikujemo *srušen* proces od *sporog*.
- **FLP:** u asinkronom sustavu konsenzus je nemoguć već uz jedan kvar.

Rješenje

Sustavu dodajemo *otkrivač grešaka* — orakl koji daje (nepouzidane) naznake o tome tko se srušio. Time konsenzus postaje rješiv.

Otkrivači grešaka

- Svaki proces p_i ima lokalni modul D_i koji prati ostale procese.
- $D_p(t)$ = skup procesa za koje p u trenutku t sumnja da su se srušili.
- Sumnja se temelji na tome koliko dugo nismo čuli neki proces.
- **Nepouzdana su:** ispravan proces može biti osumnjičen, pa kasnije maknut s popisa.

Kvalitetu opisujemo dvama svojstvima: **potpunost** i **točnost**.

Potpunost

Kada otkrivač *mora* posumnjati na srušeni proces.

Jaka potpunost

Nakon nekog trenutka svaki srušeni proces trajno je sumnjiv *svakom* ispravnom procesu.

Slaba potpunost

Nakon nekog trenutka svaki srušeni proces trajno je sumnjiv *nekom* ispravnom procesu.

Sama potpunost nije dovoljna (detektor koji sumnja na sve zadovoljava jaku potpunost, a beskoristan je).

Točnost

Kada otkrivač *smije* pogriješiti (posumnjati na ispravan proces).

- **Jaka:** nijedan ispravan proces nikada nije sumnjiv.
- **Slaba:** neki ispravan proces nikada nije sumnjiv.
- **Konačna jaka:** postoji trenutak nakon kojeg nijedan ispravan proces nije sumnjiv.
- **Konačna slaba:** postoji trenutak nakon kojeg neki ispravan proces nije sumnjiv.

„Konačna” varijanta dopušta rane greške, ali zahtijeva da one *prestanu*.

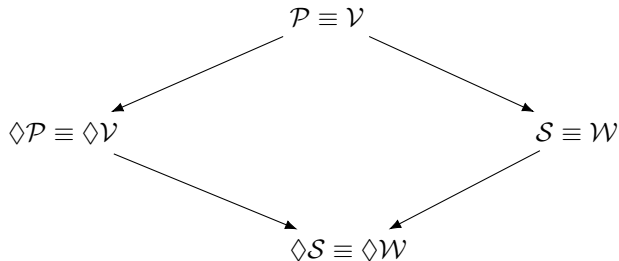
Osam klasa otkrivača

Klasa	Potpunost	Točnost	Oznaka
Savršeni	jaka	jaka	$\mathcal{P}$
Konačno savršeni	jaka	konačna jaka	$\diamond \mathcal{P}$
Jaki	jaka	slaba	$\mathcal{S}$
Konačno jaki	jaka	konačna slaba	$\diamond \mathcal{S}$
Slabi	slaba	slaba	$\mathcal{W}$
Konačno slabi	slaba	konačna slaba	$\diamond \mathcal{W}$
Slaba potpunost i jaka točnost	slaba	jaka	$\mathcal{V}$
Slaba potpunost i konačna jaka točnost	slaba	konačna jaka	$\diamond \mathcal{V}$

Reducibilnost

Transformacijom slabe potpunosti u jaku dobivamo ekvivalencije:

$$\mathcal{P} \equiv \mathcal{V}, \quad \mathcal{S} \equiv \mathcal{W}, \quad \Diamond \mathcal{P} \equiv \Diamond \mathcal{V}, \quad \Diamond \mathcal{S} \equiv \Diamond \mathcal{W}.$$



Za konsenzus je dovoljno riješiti dvije klase: $\mathcal{S}$ i $\Diamond \mathcal{S}$.

Konsenzus pomoću otkrivača

Svaki proces predlaže vrijednost; traži se *konačnost*, *usuglašenost* i *valjanost*.

Uz $\mathcal{S}$ (jaki)

Konsenzus rješiv **bez ograničenja na broj kvarova** (do $n - 1$).

⇒ Algoritam 15.3.

Uz $\diamond \mathcal{S}$ (konačno jaki)

Konsenzus rješiv **samo ako je većina ispravna** ($f < n/2$).

⇒ Algoritam 15.4.

$\diamond \mathcal{S}$ je najslabiji otkrivač koji uopće rješava konsenzus; u praksi gradimo barem $\diamond \mathcal{P}$ (Algoritam 15.7).

Algoritam 15.3 — konsenzus uz $\mathcal{S}$

Ideja. Konsenzus koji podnosi **bilo koji broj kvarova** (do $n - 1$) i ne treba većinu; radi u tri faze.

- Svaki proces drži vektor V_p — svoje procjene vrijednosti koje su predložili svi procesi (na početku prazne, osim vlastite).
- Δ_p sadrži vrijednosti naučene u zadnjoj rundi.
- Upit „ $q \in D_p$ ” (koga sumnjičimo) ostvaruje ugrađeni heartbeat detektor, poruke „Alive”.

Algoritam 15.3 — opis

- **Faza 1** ($n - 1$ rundi). U svakoj rundi proces svima pošalje vrijednosti koje je zadnje naučio i čeka poruku te runde od svakog procesa kojeg *ne* sumnjiči — osumnjičene preskače. Svaku novu vrijednost upiše u V_p . Nakon $n - 1$ rundi svi ispravni imaju isti skup „preživjelih” vrijednosti.
- **Faza 2.** Procesi razmijene cijele vektore. Ako je bilo koji primljeni vektor imao prazno mjesto, i vlastito se mjesto isprazni — tako ispadaju vrijednosti oko kojih nema potpune sloge. Barem jedno mjesto ostaje popunjeno.
- **Faza 3.** Svi odluče *prvu* popunjenu vrijednost vektora; ona je ista kod svih.

Algoritam 15.3 — kod I

```
for(int runda=1; runda < numProc; runda++){  
    // Faza 1: posalji svoju Deltu svima  
    for (int j = 0; j < numProc; j++)  
        if (j != myId) process.sendMessage(j,"Consensus",runda,process.getDelta())  
        ;  
  
    // cekaj dok ne primis od svih ili ih ne osumnjicis (upit detektoru)  
    do {  
        for (int j = 0; j < numProc; j++)  
            if (j != myId){ process.sendMessage(j,"Alive",runda,process.getDelta())  
                };  
                process.provjeriProces(j); }  
        Thread.sleep(1000);  
    } while(process.kolikoSamPorukaPrimioURundi(runda)  
        + process.kolikoJeProcesaOsumnjiceno() < numProc-1);  
  
    process.obradiPoruke(runda);    // spoji primljene vrijednosti u V  
    process.novaRunda();  
}
```

Algoritam 15.3 — kod II

```
// Faza 1: za svako mjesto i koje je jos null, uzmi vrijednost iz poruke
public void obradiPoruke(int runda){
    for (int i = 0; i < numProc; i++) Delta.set(i, null);
    for (int i = 0; i < numProc; i++){
        if(V.get(i)==null){
            for(int j = 0; j < Poruke.get(runda-1).size(); j++){
                String[] t = Poruke.get(runda-1).get(j).getMessage().trim().
                    split("\\s+");
                Integer val = t[i+1].equals("null") ? null : Integer.parseInt(
                    t[i+1]);
                if(val!=null){ V.set(i,val); Delta.set(i,val); }
            }
        }
    }
}

// Faza 2: ako je neki primljeni V_q imao null na mjestu i, i mi ga ponistimo
public void provjeriZadnjePoruke(int proces){
    for(int i = 0; i<ZadnjePoruke.size(); i++){
        if(ZadnjePoruke.get(i).getSrcId()==proces){
```

Algoritam 15.3 — kod III

```
String[] t = ZadnjePoruke.get(i).getMessage().trim().split("\\s+");  
    ;  
    Integer val = t[i+1].equals("null") ? null : Integer.parseInt(t[i  
        +1]);  
    if(val==null) V.set(i, null);  
    }  
}  
}
```

Algoritam 15.3 — zašto radi

- **Otpornost:** nijedna faza ne traži većinu — čeka se svaki neosumnjičeni proces, osumnjičeni se preskaču.
- **Usuglašenost:** slaba točnost $\mathcal{S}$ jamči ispravan proces p^* kojeg nitko ne sumnjiči; njegova vrijednost preživi u svim vektorima $\Rightarrow$ svi odluče isto.
- **Konačnost:** jaka potpunost $\Rightarrow$ srušeni se na kraju osumnjiče, pa čekanje ne blokira zauvijek.

Algoritam 15.4 — konsenzus uz $\diamond\mathcal{S}$

Ideja. Konsenzus *rotirajućim koordinatorom*: u rundi r koordinator je $c = (r \bmod n) + 1$, i sve poruke idu prema koordinatoru ili od njega.

- Traži **ispravnu većinu** procesa ($f < n/2$).
- Svaki proces pamti procjenu odluke $estimate_p$ i vremensku oznaku ts_p (rundu u kojoj je procjenu zadnji put promijenio).
- Sumnju na koordinatora daje otkrivač $\diamond\mathcal{S}$.

Algoritam 15.4 — opis

U svakoj rundi koordinator pokušava „zaključati” zajedničku vrijednost kroz četiri faze:

- **Faza 1.** Svi procesi pošalju koordinatoru svoju procjenu i oznaku ($estimate_p, ts_p$).
- **Faza 2.** Koordinator pričekava procjene od većine, odabere najsvježiju (najveća oznaka ts) i pošalje je svima kao prijedlog.
- **Faza 3.** Tko primi prijedlog, prihvati ga i odgovori *ack*; tko posumnja na koordinatora, odgovori *nack*.
- **Faza 4.** Dobije li koordinator *ack* od većine, vrijednost je zaključana i emitira *decide*; svi koji ga prime odluče tu vrijednost.

Sruši li se koordinator ili je osumnjičen, kreće iduća runda s novim koordinatorom.

Algoritam 15.4 — kod I

```
public void runConsensus(){
    while(state == 0){
        receivedPhase1Messages.clear();
        receivedPhase2Messages.clear();
        round++;
        int coordinatorId = (round%n);
        phase1_sendEstimate(coordinatorId);
        phase2_coordinatorSelect(coordinatorId);
        phase3_ackNack(coordinatorId);
        phase4_decide(coordinatorId);
        try { Thread.sleep(1000); } catch (InterruptedException e) {}
    }
    System.out.println("FINAL DECISION = " + estimate);
}
```

Algoritam 15.4 — kod II

```
public void phase1_sendEstimate(int coordinatorId) {
    String payload = estimate + " " + estimateRound;    // estimate + timestamp
    if (coordinatorId != myId)
        linker.sendMsg(coordinatorId, "PHASE1", payload);
}

public void phase2_coordinatorSelect(int coordinatorId) {
    if (myId != coordinatorId) return;                // samo koordinator
    int bestEstimate = estimate, bestRound = estimateRound;
    int majority = (n/2) + 1;
    while(receivedPhase1Messages.size() < majority-1)
        try { Thread.sleep(100); } catch (InterruptedException e){ break; }
    for (Msg m : new ArrayList<>(receivedPhase1Messages)) {
        String[] parts = m.getMessage().trim().split("\\s+");
        int value = Integer.parseInt(parts[0]), ts = Integer.parseInt(parts
            [1]);
        if (ts > bestRound) { bestRound = ts; bestEstimate = value; }    //
            najsvjeziji
    }
    estimate = bestEstimate; estimateRound = round;
}
```

Algoritam 15.4 — kod III

```
linker.broadcastToAll("PHASE2", estimate + " " + estimateRound);
receivedPhase2Messages.add(new Msg(myId, myId, "PHASE2", estimate+" "+
    estimateRound));
}
```

```
public void phase3_ackNack(int coordinatorId) {
    Msg coordinatorMsg = null;
    while(receivedPhase2Messages.isEmpty())
        try { Thread.sleep(100); } catch(InterruptedException e){}
    for (Msg m : new ArrayList<>(receivedPhase2Messages)) {
        if (m.getSrcId()==coordinatorId && m.getDestId()==myId) {
            coordinatorMsg = m;
            String[] parts = m.getMessage().trim().split("\\s+");
            estimate = Integer.parseInt(parts[0]); estimateRound = round; //
                prihvati
            break;
        }
    }
    if (coordinatorMsg != null || coordinatorId == myId) { // ACK
```

Algoritam 15.4 — kod IV

```
        if (coordinatorId != myId) linker.sendMsg(coordinatorId, "ACK", round
            + "");
        else receivedPhase3Replies.add(new Msg(myId, myId, "ACK", round + ""))
            ;
    } else {
        if (coordinatorId != myId) linker.sendMsg(coordinatorId, "NACK", round
            + "");
    }
}

public void phase4_decide(int coordinatorId) {
    if (myId != coordinatorId) return;           // samo koordinator odlucuje
    int ackCount = 0, majority = (n/2) + 1;
    while(receivedPhase3Replies.size() < majority)
        try { Thread.sleep(100); } catch(InterruptedException e){}
    for (Msg m : new ArrayList<>(receivedPhase3Replies))
        if (m.getSrcId() != coordinatorId && m.getTag().equals("ACK")) ackCount
            ++;
    if (ackCount >= majority) {
        linker.broadcastToAll("DECIDE", estimate + " " + round);
        state = 1;
    }
}
```

Algoritam 15.4 — kod V

```
    }  
    receivedPhase3Replies.clear();  
}
```

```
// obrada odluke u handleMessage  
else if (tag.equals("DECIDE")) {  
    String[] parts = m.getMessage().trim().split("\\s+");  
    estimate = Integer.parseInt(parts[0]);  
    state = 1;  
}
```

Algoritam 15.4 — zašto radi

- **Zašto većina:** faze 2 i 4 čekaju $\lceil (n+1)/2 \rceil$ odgovora; dvije se većine uvijek preklapaju $\Rightarrow$ zaključana vrijednost prenosi se u sve kasnije runde (usuglašenost).
- Bez ispravne većine čekanje bi moglo blokirati $\Rightarrow$ uvjet $f < n/2$.
- **Konačnost:** konačna slaba točnost $\diamond \mathcal{S} \Rightarrow$ na kraju postoji ispravan, neosumnjichen koordinator koji zaključa vrijednost i emitira *decide*.

Algoritam 15.7 — otkrivač $\diamond\mathcal{P}$

Ideja. Savršen otkrivač $\mathcal{P}$ nije ostvariv u asinkronom sustavu; najbolje što možemo jest da greške *na kraju* prestanu — to je $\diamond\mathcal{P}$.

- Radi u *parcijalnoj sinkroniji* (nakon nepoznatog trenutka sustav postane sinkron).
- Vrijeme se mjeri lokalnim satom (otkucaji), ne stvarnim satom.
- Ključ: **adaptivni timeout** $\Delta_p(q)$ koji raste nakon svake pogrešne sumnje.

Algoritam 15.7 — opis (tri zadatka)

Tri paralelna zadatak; vrijeme mjeri lokalni sat koji broji otkucaje.

- **Zadatak 1 (heartbeat).** Proces periodički svima šalje poruku „živ sam”.
- **Zadatak 2 (timeout).** Ako od procesa q nije čuo ništa dulje od $\Delta_p(q)$ otkucaja, doda ga među sumnjive.
- **Zadatak 3 (kajanje).** Primi li poruku od procesa kojeg sumnjiči, makne ga s popisa i *poveća* $\Delta_p(q)$ za jedan.

Timeout se sam prilagođava: raste dok ne prijeđe stvarno kašnjenje, nakon čega lažne sumnje prestaju.

Algoritam 15.7 — kod I

```
// Zadatak 1: posalji "ziv sam" svima
public void sendHeartbeats() {
    if (crashed) return;
    broadcastMsg("alive", (int) clock);
}

// Zadatak 2: istek vremena -> sumnja
public synchronized void checkTimeouts() {
    for (int q = 0; q < N; q++) {
        if (q == myId) continue;
        if (!Output.contains(q) && (clock - lastAlive[q]) > delta[q]) {
            Output.add(q);
        }
    }
}

// Zadatak 3: kad primim "ziv sam" od q -> kajanje + veci timeout
public synchronized void handleMsg(Msg m, int q, String tag) {
    if (tag.equals("alive")) {
        lastAlive[q] = clock;
        if (Output.contains(q)) {
```

Algoritam 15.7 — kod II

```
        Output.remove(q);  
        delta[q] = delta[q] + 1;  
    }  
}
```

Algoritam 15.7 — ispravnost i test

Jaka potpunost. Srušeni proces prestaje slati; tišina raste iznad $\Delta_p(q) \Rightarrow$ trajno osumnjičen.

Konačna jaka točnost. Nakon sinkronosti Δ prerasta stvarno kašnjenje $\Rightarrow$ pogrešne sumnje prestaju.

Scenarij	Rezultat
kvar p_2 ($t=10$)	svi trajno sumnjaju na p_2 ; potpunost
spor p_2 (svakih 8)	Δ : $5 \rightarrow 6 \rightarrow 7 \rightarrow 8$, sumnje prestanu; točnost

Zaključak

- Otkrivači se opisuju **potpunošću** i **točnošću** $\Rightarrow$ osam klasa.
- Zbog reducibilnosti za konsenzus su bitne $\mathcal{S}$ i $\diamond \mathcal{S}$.
- **15.3** ($\mathcal{S}$): konsenzus uz bilo koji broj kvarova.
- **15.4** ($\diamond \mathcal{S}$): konsenzus uz ispravnu većinu (rotirajući koordinator).
- **15.7** ($\diamond \mathcal{P}$): ostvariv otkrivač u parcijalnoj sinkroniji (adaptivni timeout).

Hvala na pažnji!